How to Scale an App to 10 Million Users on AWS

#27: And High Scalability Explained Like You're Twelve (6 minutes)

NEO KIM

DEC 06, 2023

Get my system design playbook for FREE on newsletter signup:

Subscribed

This post outlines how to scale an app to millions of users on the cloud. If you want to learn more, scroll to the bottom and find the references.

- *Share this post & I'll send you some rewards for the referrals.*

Once upon a time, there lived 2 software engineers named James and Robert.

They worked for a tech company named Hooli.

Although they were bright engineers, they never got promoted.

So they were sad and frustrated.

Until one day when they had a smart idea to build a startup.

Their growth rate was mind-boggling.

Yet they wanted to keep it simple. So they hosted the app on AWS.

AWS Scale

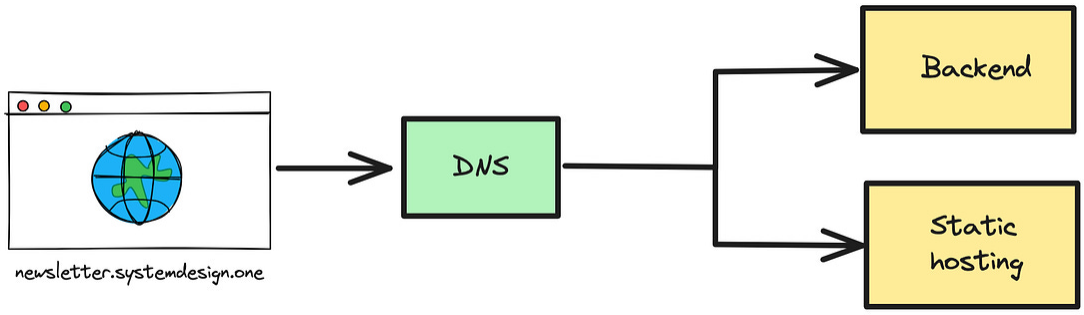
Here's their scalability journey from 0 to 10 million users:

1. Prelaunch

Their minimum viable product (**MVP**) was not production-ready.

So they used a static framework to build the launch page. And served static pages at no operational costs via AWS Amplify hosting.

AWS Amplify is a web app hosting service.



Launch Page on Static Hosting

Also they added simple backend code in AWS Lambda.

AWS Lambda is a serverless platform. It allowed them to run code without managing servers.

2. Launch

They launched the MVP.

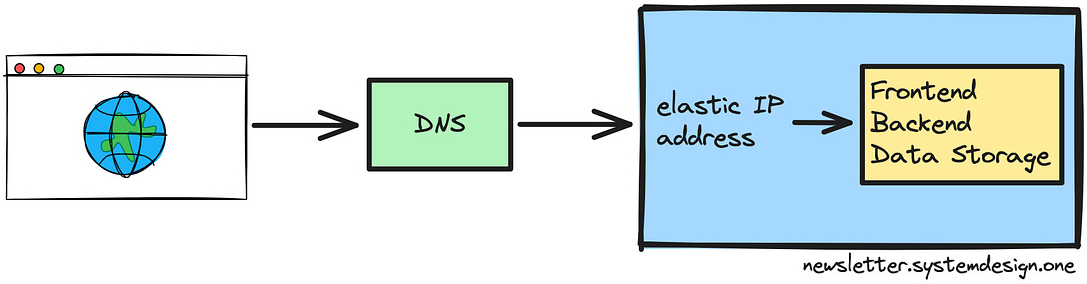
But they had only a handful of users.

So they used a single EC2 instance to run the entire web stack: database and backend.

Amazon Elastic Compute Cloud (**EC2**) is a virtual server.

They attached an elastic IP address to the EC2 instance and then pointed Amazon Route 53 *DNS* toward it.

An **Elastic IP address** is a static IP address.



Single EC2 Instance Running the Entire Web Stack

Yet they hit the storage limits because some users had more data.

So they installed a larger instance type to scale vertically.

Increasing the capacity of a single server for performance is called **Vertical Scaling**.

The configuration template for virtual servers is called **Instance Type**. For example, *nano* and *micro* instance types offer varying CPU, memory, and storage.

But there is a risk of a single point of failure with vertical scaling. Also a service cannot be scaled vertically beyond a limit.

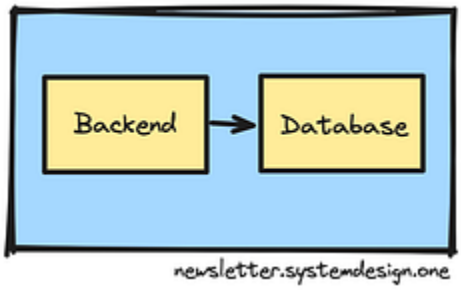
3. Ten Users

But one day.

They hit the memory limit with the single EC2 instance.

So they moved the backend and database into separate EC2 instances.

And run the database on a larger instance type.



Backend and Database Running on Separate EC2 Instances

They used an SQL database because it was enough to handle the first 10 million users. Also it offered them strong community support and tools.

And life was good.

4. Thousand Users

Until one day when they received a phone call.

There was a power outage in the availability zone hosting their servers.

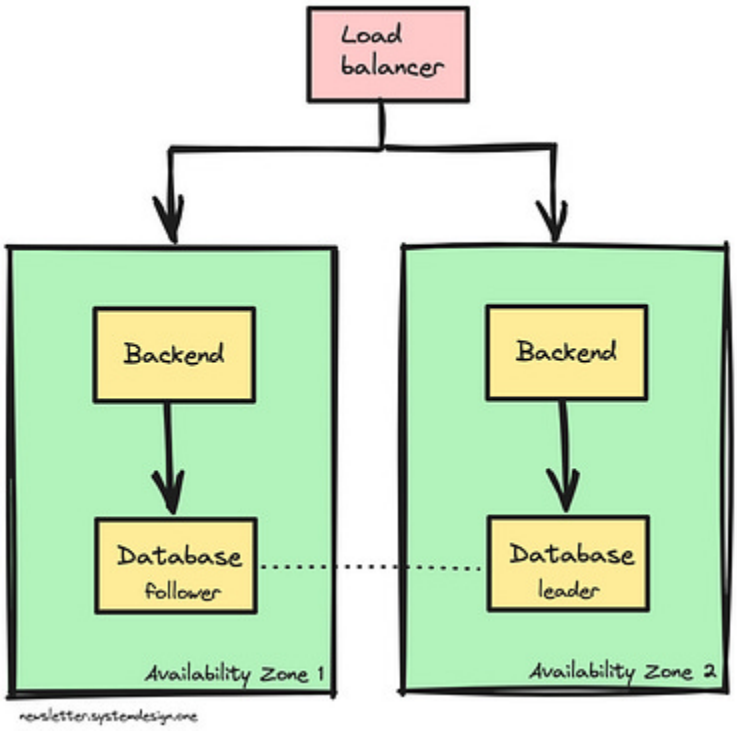
So they installed an extra server in another availability zone.

And set up the database in the leader-follower replication topology.

The database follower ran in another availability zone.

The database leader served the write operations. While the database follower served the reads.

Also they set up automatic failover for high availability.



Running the App Across Two Availability Zones

They load-balanced the traffic between 2 availability zones.

And pointed DNS toward the load balancer.

They used an Elastic Load Balancer (**ELB**). And it did server health checks to prevent traffic to failed instances.

5. Ten Thousand Users

But one day.

They hit the memory limit again. And servers ran at full capacity.

They wanted to scale out.

So they made the backend stateless by moving out the session state. And storing it in ElasticCache.

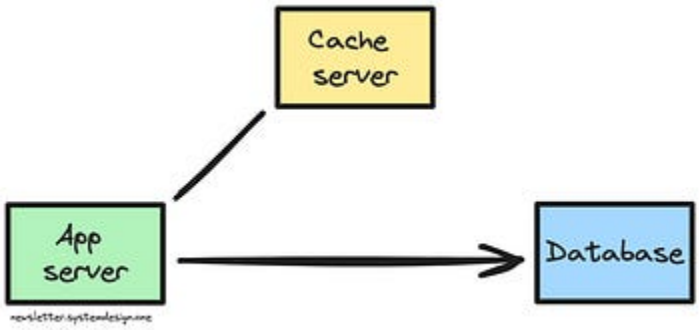
Besides they added more servers behind the load balancer to scale horizontally. And installed extra database followers.

Making App Server Stateless by Storing State in Cache

Yet the database reads were becoming a bottleneck.

So they cached the popular database reads in ElasticCache.

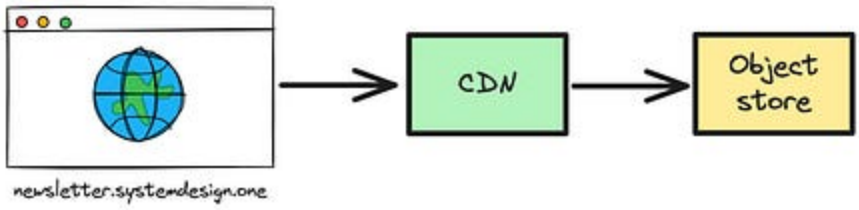
ElasticCache is a managed Memcached or Redis.



Caching Database Reads to Reduce Load

Also they moved static content to Amazon S3 and CloudFront to reduce the server load. The static content is CSS, images, and JavaScript files.

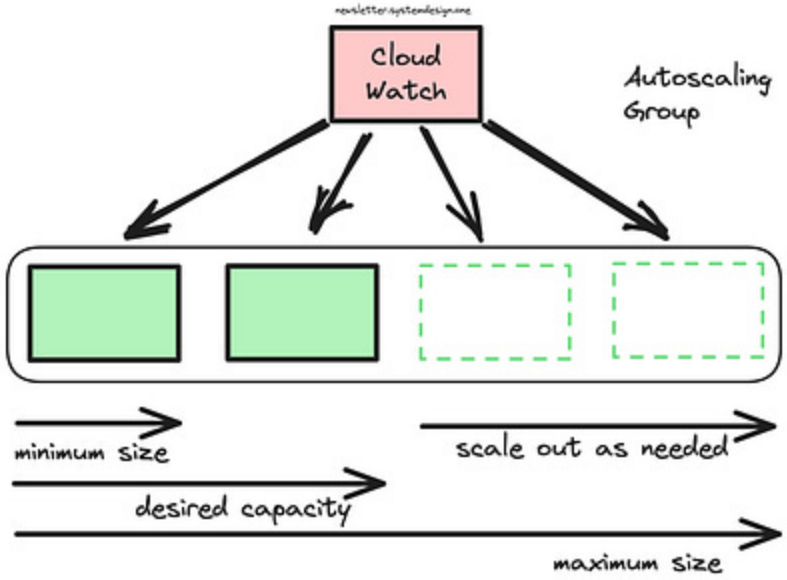
Amazon S3 is an object store. While *CloudFront* is a Content Delivery Network (**CDN**).



CDN Hosting Static Content

They reduced latency by caching data in CDN at edge locations.

A content delivery endpoint in a CDN network is called an **Edge Location.**



Dynamically Scaling Servers With Autoscaling

Yet one day.

They noticed their peak traffic was much higher than average traffic.

So they over-provisioned servers to handle peak traffic.

But it cost them a lot of money.

So they set up autoscaling with CloudWatch.

Scaling the computing resources based on load is called **Autoscaling**. It reduced their costs and operational complexity.

CloudWatch is Amazon's monitoring and management service. It embeds an agent in each service to collect metrics.

6. Half a Million Users

Their growth was inevitable.

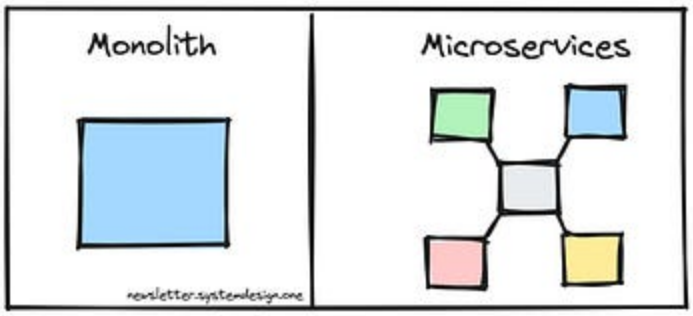
And they wanted to offer the users a service level agreement (**SLA**) of 99.99% high availability.

Put another way, they couldn't tolerate downtime of more than 52 minutes per year.

So they replicated the servers across many availability zones. And offloaded session data to DynamoDB.

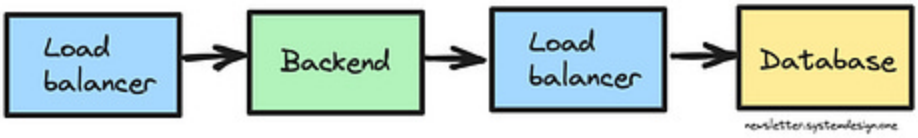
Besides they ran each server at full capacity for performance.

DynamoDB is a managed NoSQL database.



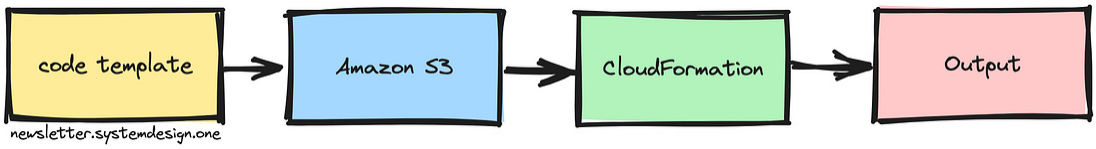
Monolith vs Microservices

They moved to a microservices architecture because wanted to scale services independently.



Load Balancing Different Layers

Also they added load balancers between each layer to scale out.



CloudFormation Workflow

They used AWS CloudFormation to create a templatized view of the web stack. Because it reduced the operational complexity.

CloudFormation is an infrastructure as a code deployment service.

And life was good again.

7. Ten Million Users

But one day.

They noticed database writes being slow due to data contention.

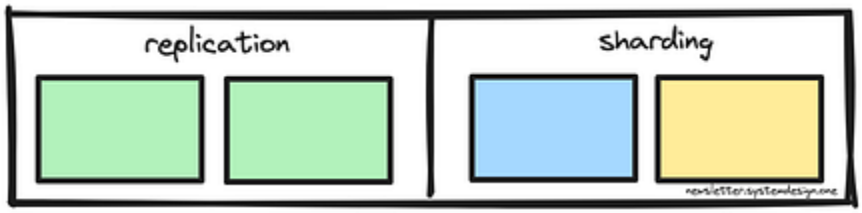
So they federated the database and then sharded it.

Database Federation

Splitting a database into many databases based on the business domain is called **Federation**. For example, storing product and user information in separate databases.

Federation allowed them to scale easily.

Yet it became difficult to query cross-function data due to many databases.



Replication vs Sharding

Splitting a single dataset across many servers is called **Sharding**.

They used sharding to scale out.

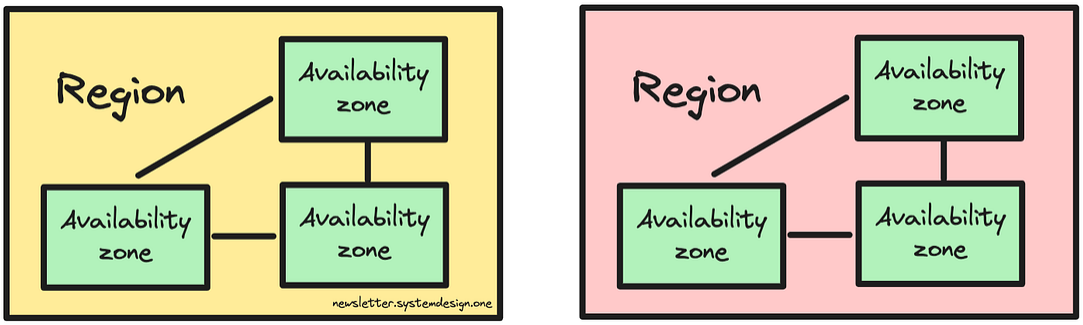
But it made the application layer more complex because the data needed to be joined at the application level.

Also they moved the data that doesn't need complex joins to a NoSQL database.

Besides they extended the system across many AWS regions. Because it offered high availability and scalability.

And lived happily ever after.

AWS has 32 regions and 102 availability zones around the world.



AWS Regions and Availability Zones

A Geographical area with data centers is called an **AWS Region**. Each region has more than a single availability zone.

A single data center or a combination of many data centers within an AWS region is called an **Availability Zone**.

Each AZ is connected to a separate network and power for high availability.

Consider subscribing to get simplified case studies delivered straight to your inbox:

Subscribed

Follow me on [LinkedIn](#) | [YouTube](#) | [Threads](#) | [Twitter](#) | [Instagram](#)

Thank you for supporting this newsletter. Consider sharing this post with your friends and get rewards. Y'all are the best.

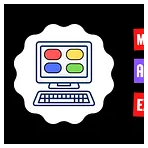
Share

How Uber Computes ETA at Half a Million

Requests per Second

NK · DECEMBER 3, 2023

Read full story



Everything You Need to Know About Micro Frontends

NK · NOVEMBER 14, 2023

Read full story

References

- Scaling on AWS for your first 10 million users
- AWS Documentation Overview
- SRE fundamentals